



TESTING

Las pruebas de software (en inglés software testing) son las investigaciones empíricas y técnicas cuyo objetivo es proporcionar información objetiva e independiente sobre la calidad del producto a la parte interesada o stakeholder. Es una actividad más en el proceso de control de calidad.

Las diversas pruebas a que debe ser sometido un sistema deben ser realizadas tanto por el equipo de desarrolladores, como por los usuarios, equipos de operación y mantenimiento en la implantación, aceptación y mantenimiento del sistema de información.

Por ejemplo, en la metodología Métrica v3 los tipos de pruebas que deben realizarse son:

- Pruebas Unitarias.
- Pruebas de Integración.
- Pruebas del Sistema.
- Pruebas de Implantación.
- Pruebas de Aceptación.
- Pruebas de Regresión.

A continuación, se van a explicar algunas de las clasificaciones más relevantes de las pruebas de software.

PRUEBAS DE CAJA BLANCA Y CAJA NEGRA

Pruebas de "Caja Blanca": el esfuerzo se concentra en encontrar casos de prueba que permitan comprobar cómo funciona el código en cualquier situación, lo que supone, como mínimo, recorrer todos sus caminos con un conjunto tan amplio de datos de prueba como sea necesario. En resumen, se trataría de recorrer exhaustivamente todos los caminos posibles.

Pruebas de "Caja Negra": consiste en visualizar el programa que se desea probar como una caja negra de forma que quien realiza las pruebas no se preocupa en absoluto por su contenido y concentra sus esfuerzos en encontrar circunstancias en las que el programa no se comporte de acuerdo con las especificaciones. Se intenta probar completamente en esta aproximación:

- Las funciones realizadas por el sistema.
- El cumplimiento de los objetivos del sistema.
- Las reacciones del sistema ante los estímulos exteriores.
- Las transacciones manejadas por el sistema.

PRUEBAS ESTÁTICAS Y DINÁMICAS

Pruebas estáticas: Son el tipo de pruebas que se realizan sin ejecutar el código de la aplicación. Puede referirse a la revisión de documentos, ya que no se hace una ejecución de código. Esto se debe a que se pueden realizar "pruebas de escritorio" con el objetivo de seguir los flujos de la aplicación.

Pruebas dinámicas: son todas aquellas pruebas que para su ejecución requieren la ejecución de la aplicación. Las pruebas dinámicas permiten el uso de técnicas de caja negra y caja blanca con mayor amplitud. Debido a la naturaleza dinámica de la ejecución de pruebas es posible medir con mayor precisión el comportamiento de la aplicación desarrollada.



PRUEBAS FUNCIONALES Y NO FUNCIONALES

Pruebas funcionales: Una prueba funcional es una prueba basada en la ejecución, revisión y retroalimentación de las funcionalidades previamente diseñadas para el software (requisitos funcionales). Hay distintos tipos, como por ejemplo:

- **Pruebas unitarias:** tienen como objetivo verificar la funcionalidad y estructura de cada componente individualmente una vez que ha sido codificado.
- **Pruebas de integración:** consisten en verificar el correcto ensamblaje entre los distintos componentes una vez que han sido probados unitariamente con el fin de comprobar que interactúan correctamente a través de sus interfaces, tanto internas como externas, cubren la funcionalidad establecida y se ajustan a los requisitos no funcionales especificados en las verificaciones correspondientes.
- **Pruebas de sistema:** tienen como objetivo ejercitar profundamente el sistema comprobando la integración del sistema de información globalmente, verificando el funcionamiento correcto de las interfaces entre los distintos subsistemas que lo componen y con el resto de sistemas de información con los que se comunica.
- **Pruebas alpha:** son realizadas por el usuario con el desarrollador como observador en un entorno controlado.
- **Pruebas beta:** son realizadas por el usuario en su entorno de trabajo y sin observadores.
- **Pruebas de aceptación:** su objetivo es validar que un sistema cumple con el funcionamiento esperado y permitir al usuario de dicho sistema que determine su aceptación, desde el punto de vista de su funcionalidad y rendimiento. Las pruebas de aceptación son definidas por el usuario del sistema y preparadas por el equipo de desarrollo, aunque la ejecución y aprobación final corresponden al usuario.
- **Pruebas de regresión:** consisten en comprobar que los cambios sobre un componente de un sistema de información, no introducen un comportamiento no deseado o errores adicionales en otros componentes no modificados. Las pruebas de regresión se deben llevar a cabo cada vez que se hace un cambio en el sistema, tanto para corregir un error como para realizar una mejora.

Pruebas no funcionales: Una prueba no funcional es una prueba cuyo objetivo es la verificación de un requisito que especifica criterios que pueden usarse para juzgar la operación de un sistema (requisitos no funcionales) como por ejemplo la disponibilidad, accesibilidad, usabilidad, mantenibilidad, seguridad, rendimiento. Se pueden clasificar las pruebas no funcionales según el tipo de requisito no funcional que abarcan:

- Pruebas de compatibilidad
- Pruebas de seguridad
- Pruebas de carga
- Pruebas de usabilidad
- Pruebas de rendimiento
- Pruebas de internacionalización y localización
- Pruebas de escalabilidad
- Pruebas de mantenibilidad
- Pruebas de instalabilidad
- Pruebas de portabilidad



DESARROLLO GUIADO POR PRUEBAS (TDD)

Desarrollo guiado por pruebas de software, o Test-driven development (TDD) es una práctica de ingeniería de software que involucra otras dos prácticas:

- Escribir las pruebas primero (Test First Development) basándose en los requisitos y se verifica que las pruebas fallen.
- Escribir el código fuente que pase la prueba satisfactoriamente.
- Refactorización el código escrito (Refactoring).

Para escribir las pruebas generalmente se utilizan las pruebas unitarias. En primer lugar, se escribe una prueba y se verifica que las pruebas fallan. A continuación, se implementa el código que hace que la prueba pase satisfactoriamente y seguidamente se refactoriza el código escrito.

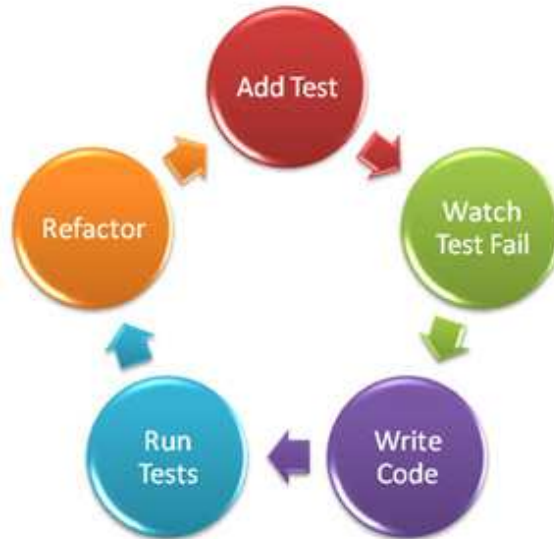
El propósito del desarrollo guiado por pruebas es lograr un código limpio que funcione. La idea es que los requisitos sean traducidos a pruebas, de este modo, cuando las pruebas pasen se garantizará que el software cumple con los requisitos que se han establecido.

Ciclo de desarrollo conducido por pruebas

En primer lugar se debe definir una lista de requisitos y después se ejecuta el siguiente ciclo:

1. **Elegir un requisito:** Se elige de una lista el requisito que se cree que nos dará mayor conocimiento del problema y que a la vez sea fácilmente implementable.
2. **Escribir una prueba:** Se comienza escribiendo una prueba para el requisito. Para ello el programador debe entender claramente las especificaciones y los requisitos de la funcionalidad que está por implementar. Este paso fuerza al programador a tomar la perspectiva de un cliente considerando el código a través de sus interfaces.
3. **Verificar que la prueba falla:** Si la prueba no falla es porque el requisito ya estaba implementado o porque la prueba es errónea.
4. **Escribir la implementación:** Escribir el código más sencillo que haga que la prueba funcione.
5. **Ejecutar las pruebas automatizadas:** Verificar si todo el conjunto de pruebas funciona correctamente.
6. **Eliminación de duplicación:** El paso final es la refactorización, que se utilizará principalmente para eliminar código duplicado. Se hace un pequeño cambio cada vez y luego se corren las pruebas hasta que funcionen.
7. **Actualización de la lista de requisitos:** Se actualiza la lista de requisitos tachando el requisito implementado. Asimismo se agregan requisitos que se hayan visto como necesarios durante este ciclo y se agregan requisitos de diseño (P. ej que una funcionalidad esté desacoplada de otra).

Tener un único repositorio universal de pruebas facilita complementar TDD con otra práctica recomendada por los procesos ágiles de desarrollo, la "Integración Continua". Integrar continuamente nuestro trabajo con el del resto del equipo de desarrollo permite ejecutar toda batería de pruebas y así descubrir si nuestra última versión es compatible con el resto del sistema.



Ventajas

- A pesar de los elevados requisitos iniciales de aplicar esta metodología, el desarrollo guiado por pruebas (TDD) puede proporcionar un gran valor añadido en la creación de software, produciendo aplicaciones de más calidad y en menos tiempo.
- Ofrece más que una simple validación del cumplimiento de los requisitos, también puede guiar el diseño de un programa.
- Al hacer primero las pruebas se realiza un ejercicio previo de análisis en profundidad de los requisitos y de los diversos escenarios.
- El hecho que sólo se implemente el código necesario para resolver un caso de prueba, hace que el código creado sea el mínimo necesario, reduciendo redundancia y bloques de código de "por si a caso".



DESARROLLO GUIADO POR COMPORTAMIENTO (BDD)

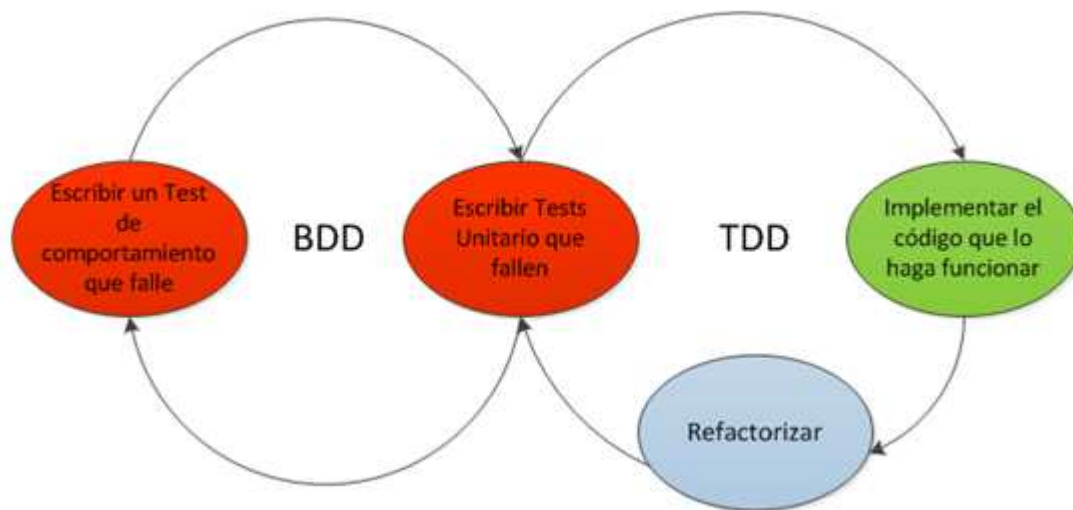
El Desarrollo Dirigido por Comportamiento (Behaviour-driven development o BDD) es una técnica de desarrollo ágil orientada al negocio y comportamiento de una aplicación. Su principal objetivo es preguntar qué debería hacer la aplicación antes y durante el proceso de desarrollo. Esto se consigue mediante tests de comportamiento o especificaciones, que son escritas por los desarrolladores y que a continuación se encargan de implementar.

BDD parte de la práctica de Programación Desarrollo Orientado a Pruebas (TDD) y comparte sus mismas fases:

- Escribir un test y comprobar qué falla.
- Desarrollar el mínimo código necesario para que el test pase.
- Refactorizar.

Mientras que TDD se centra en pruebas unitarias, BDD se basa en escribir pruebas que verifiquen que el comportamiento del código es correcto desde el punto de vista de negocio. Su propósito es definir el comportamiento de la aplicación independientemente de cómo esté implementado. Por ello, BDD y TDD no sólo son compatibles, sino que es muy recomendable su uso conjunto.

Tras determinar las pruebas, se escribe el código fuente de la funcionalidad que haga que estas pruebas pasen correctamente. Después, se refactoriza el código fuente.



Para empezar a hacer BDD sólo hace falta conocer 5 palabras, con las que se construirán sentencias con las que se van a describir las funcionalidades:

- **Característica:** Indica el nombre de la funcionalidad que vamos a probar. Debe ser un título claro y explícito. Se parte de una descripción en forma de historia de usuario: **"Como [rol] quiero [característica] para que [los beneficios]"**. A partir de aquí, en lugar de describir en 'lenguaje natural' lo que tiene que hacer esa nueva funcionalidad, se va a usar un lenguaje que va a permitir describir todas las funcionalidades de una misma forma, un lenguaje específico para BDD. Sobre esta descripción ese empiezan a construir los escenarios de prueba.
- **Escenario:** Describe cada escenario que se va a probar.
- **Dado:** Provee contexto para el escenario en que se va a ejecutar el test, tales como punto donde se ejecuta el test, o prerequisites en los datos. Incluye los pasos necesarios para poner al sistema en el estado que se desea probar.



- **Cuando:** Especifica el conjunto de acciones que lanzan el test. La interacción del usuario que acciona la funcionalidad que deseamos testear.
- **Entonces:** Especifica el resultado esperado en el test. Se observan los cambios en el sistema y se comprueba si son los deseados.

Lo normal es probar distintos escenarios para comprobar una determinada funcionalidad. De esta forma, se pasa de las historias de usuario a pruebas de comportamiento automatizables.

BDD - Gherkin

Story card

User story

As a <role>

I want to <feature>

So that <benefit>

Acceptance criteria

Scenario

Given <initial context>

When <event occurs>

Then <ensure some outcomes>

Example:

Feature: Customer withdraws cash

As a customer,

I want to withdraw cash from an ATM,

So that I don't have to wait in line at the bank

Scenario: Account is in credit

Given the account is in credit

And the card is valid

And the dispenser contains cash

When the customer requests cash

Then ensure the account is debited

And ensure cash is dispensed

And ensure the card is returned

Scenario: Account is overdrawn past overdraft limit

Given the account is overdrawn

And the card is valid

When the customer requests cash

Then ensure a rejection message is displayed

And ensure cash is not dispensed

And ensure the card is returned

Ejemplo de BDD

HERRAMIENTAS PARA AUTOMATIZAR PRUEBAS EN BDD

- **Cucumber:** es una de las herramientas que podemos utilizar para automatizar nuestras pruebas en BDD. Nos va permitir ejecutar descripciones funcionales en texto plano como pruebas de software automatizadas. Está escrito en Ruby, aunque tiene implementaciones para casi cualquier lenguaje de programación.
- **Concordion, easyb y JDave** para Java.
- **Jasmine** para Javascript.
- **Kahlan** para PHP.
- **Behave** para Python.



HERRAMIENTAS PARA LA REALIZACIÓN DE PRUEBAS

Herramientas Open Source:

1 Herramientas de gestión de pruebas

- Bugzilla Testopia
- FitNesse
- qaManager
- qaBook
- RTH (open source)
- Salome-tmf
- Squash TM
- Test Environment Toolkit
- TestLink
- Testitool
- XQual Studio
- Radi-testdir
- Data Generator

2 Herramientas para pruebas funcionales

- Selenium
- Soapui
- Watir (Pruebas de aplicaciones web en Ruby)
- WatiN (Pruebas de aplicaciones web en .Net)
- Capedit
- Canoo WebTest
- Solex
- Imprimatur
- SAMIE
- ITP
- WET
- WebInject

3 Herramientas para pruebas de carga y rendimiento

- FunkLoad
- FWPTT load testing
- loadUI
- JMeter

4 Entre los frameworks para realizar pruebas unitarias

- XUnit: JUnit, NUnit, RUnit, PHPUnit...
- TestNG: Creado para suplir algunas deficiencias en JUnit.
- JTiger: Basado en anotaciones, como TestNG.
- CPPUnit
- Visual Studio UnitTesting



Herramientas comerciales:

1 Herramientas de gestión de pruebas

- HP Quality Center/ALM
- QA Complete
- qaBook
- T-Plan Professional
- SMARTS
- QAS.Test Case Studio
- PractiTest
- SpiraTest
- TestLog
- ApTest Manager
- Zephyr

2 Herramientas para pruebas funcionales

- QuickTest Pro
- Rational Robot
- Sahi
- SoapTest
- Test Complete
- QA Wizard
- Squish
- vTest
- Internet Macros

3 Herramientas para pruebas de carga y rendimiento

- HP LoadRunner
- LoadStorm
- NeoLoad
- WebLOAD Professional
- Forecast
- ANTS – Advanced .NET Testing System
- Webserver Stress Tool
- Load Impact

Finalmente, se recomienda revisar la ficha relativa a integración continua, por estar íntimamente relacionadas.